TASKFLOW · ASP.NET CORE 8 · MYSQL · JWT

Login and Registration Backend Guide

How the TaskFlow API registers users, stores passwords safely, logs users in, issues JSON Web Tokens and checks them on every request. Every code sample comes from the project's source.

Contents

1. The big picture
2. The files involved
3. The User table and roles
4. Request and response shapes (DTOs)
5. Input validation
6. Registration, step by step
7. Password hashing with BCrypt
8. Login, step by step
9. Creating the JWT
10. Checking the token on every request
11. The /me endpoint
12. Errors and status codes
13. Security checklist and limitations
14. Testing it yourself
15. Interview questions

Project folder: `backend/TaskManager.API` · Diagrams: 8 · For interview preparation

PART 1

The big picture

Login and registration answer one question: **who is this user?** This is called **authentication**. TaskFlow uses **stateless JWT authentication**. The server does not keep sessions. After a successful login it gives the client a signed token, and the client sends that token with every later request.

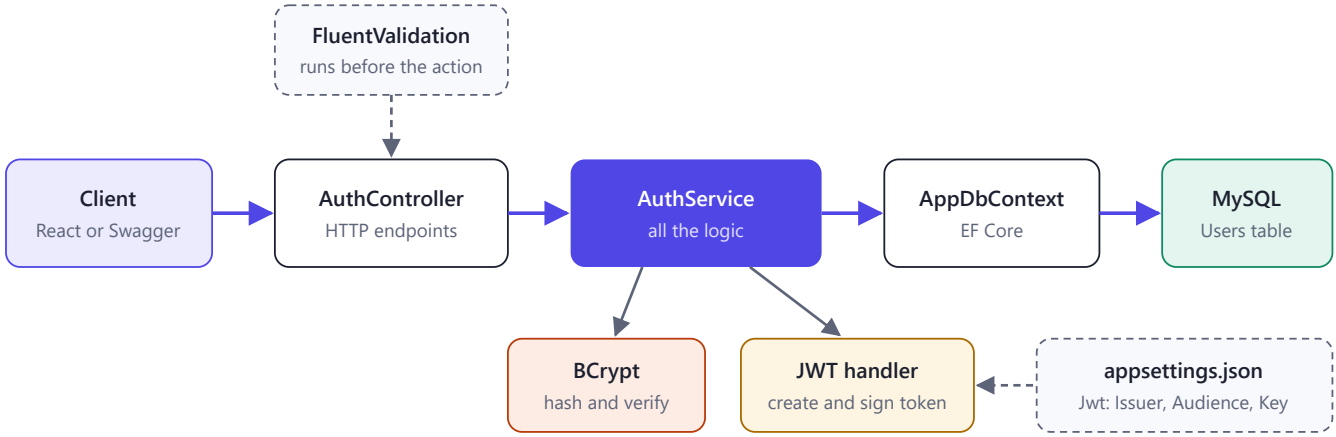


Figure 1. The parts that take part in registration and login. AuthService does the real work; the controller only receives HTTP requests.

Three endpoints

METHOD	ROUTE	NEEDS TOKEN?	WHAT IT DOES
`POST`	`/api/auth/register`	No	Creates an account and returns a token, so the user is logged in straight away
`POST`	`/api/auth/login`	No	Checks email and password and returns a token
`GET`	`/api/auth/me`	Yes	Returns the user who owns the token

Authentication vs authorization

	AUTHENTICATION	AUTHORIZATION
Question	Who are you?	What are you allowed to do?
In TaskFlow	Login, register, JWT validation	`[Authorize]`, roles, owner and member checks
Failure code	**401** Unauthorized	**403** Forbidden

This guide is about **authentication**. Authorization builds on it: it reads the user id and role that authentication puts into the request.

PART 2

The files involved

FILE	JOB
`Models/User.cs`	The User entity and the `Roles` constants
`DTOs/AuthDtos.cs`	Shapes of requests and responses
`Validators/Validators.cs`	Rules for register and login input
`Controllers/AuthController.cs`	The three HTTP endpoints
`Services/AuthService.cs`	Register, login, token creation, current user
`Extensions/ClaimsPrincipalExtensions.cs`	`User.GetUserId()` and `User.IsAdmin()` helpers
`Exceptions/AppExceptions.cs`	`BadRequestException`, `UnauthorizedException`, …
`Middleware/ExceptionMiddleware.cs`	Turns those exceptions into JSON errors
`Program.cs`	Registers JWT authentication and sets the middleware order
`appsettings.json`	JWT issuer, audience, secret key, lifetime

PART 3

The User table and roles

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;   // never the real password
    public string Role { get; set; } = Roles.Member;           // new users are Members
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    ...
}

public static class Roles
{
    public const string Admin = "Admin";
    public const string Member = "Member";
}
```

COLUMN	MYSQL TYPE	NOTES
`Id`	`int` , PK, auto increment	Goes into the token as the user id
`Name`	`varchar(100)`	Required
`Email`	`varchar(150)` , **unique index**	Stored in lower case; the index blocks duplicates
`PasswordHash`	`longtext`	BCrypt hash, 60 characters
`Role`	`varchar(20)`	`Admin` or `Member`

Why constants? Writing `Roles.Admin` instead of the text `"Admin"` everywhere means a typo becomes a compile error, not a silent security bug.

PART 4

Request and response shapes (DTOs)

DTOs (Data Transfer Objects) are the exact shapes that travel over HTTP. The API never sends the `User` entity itself, so `PasswordHash` can never leak.

```
public record RegisterDto(string Name, string Email, string Password);
public record LoginDto(string Email, string Password);
public record UserDto(int Id, string Name, string Email, string Role);
public record AuthResponseDto(string Token, DateTime ExpiresAt, UserDto User);
```

Request: register

```
{
  "name": "Kasun Perera",
  "email": "Kasun@Mail.com",
  "password": "Pass1234"
}
```

Response: 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "expiresAt": "2026-10-26T13:00:00Z",
  "user": { "id": 5, "name": "Kasun Perera",
            "email": "kasun@mail.com",
            "role": "Member" }
}
```

Notice: `RegisterDto` has no `Role` field. Even if someone sends `"role": "Admin"`, it is ignored. The server always decides the role.

PART 5

Input validation

FluentValidation checks the input **before** the controller runs. It is switched on in `Program.cs` with `AddFluentValidationAutoValidation()`. If a rule fails, the API returns **400** and the controller code never executes.

```
public class RegisterDtoValidator : AbstractValidator<RegisterDto>
{
    public RegisterDtoValidator()
    {
        RuleFor(x => x.Name).NotEmpty().MaximumLength(100);
        RuleFor(x => x.Email).NotEmpty().EmailAddress().MaximumLength(150);
        RuleFor(x => x.Password).NotEmpty().MinimumLength(6)
            .Matches("[A-Z]").WithMessage("Password must contain an uppercase letter.")
            .Matches("[0-9]").WithMessage("Password must contain a number.");
    }
}
```

FIELD	REGISTER RULES	LOGIN RULES
`Name`	Required, at most 100 characters	Not sent
`Email`	Required, valid email, at most 150	Required, valid email
`Password`	Required, 6+ characters, one uppercase letter, one number	Required only

Why are login rules weaker? Login only checks the input is present. Password strength rules apply when a password is created. Old accounts with older rules must still be able to log in.

PART 6

Registration, step by step

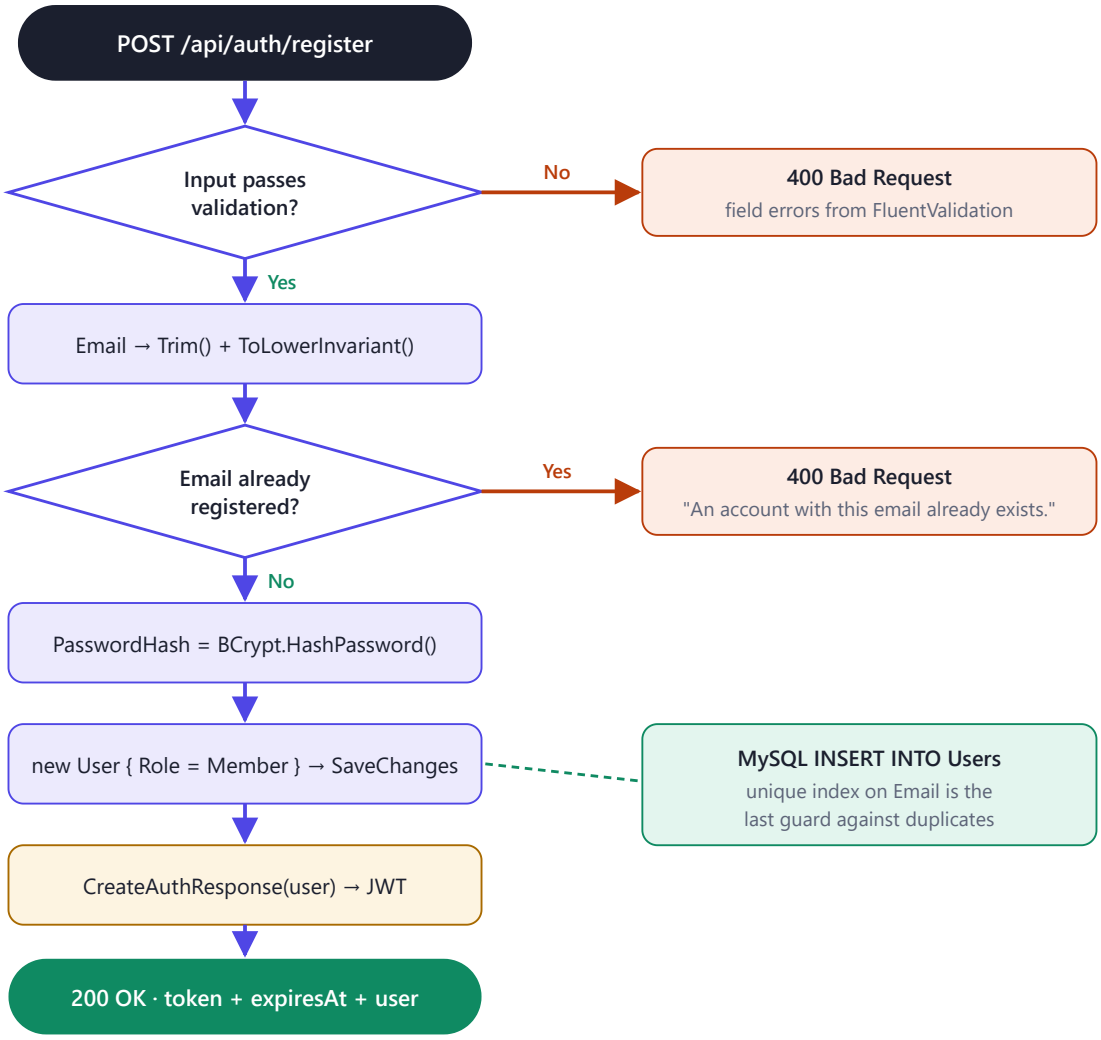


Figure 2. Registration flow. Red boxes end the request with an error; green ends it with a token.

The controller

```
[ApiController]
[Route("api/[controller]")]                // → /api/auth
public class AuthController : ControllerBase
{
    private readonly IAuthService _auth;
    public AuthController(IAuthService auth) => _auth = auth;   // dependency injection

    [HttpPost("register")]
    public async Task<ActionResult<AuthResponseDto>> Register(RegisterDto dto) =>
        Ok(await _auth.RegisterAsync(dto));
    ...
}
```

The controller is one line per endpoint. It has no `try/catch`: errors are thrown by the service and handled by the exception middleware (Part 12). There is no `[Authorize]` on register and login, because the user has no token yet.

The service

```
public async Task<AuthResponseDto> RegisterAsync(RegisterDto dto)
{
    var email = dto.Email.Trim().ToLowerInvariant();                      // 1
    if (await _db.Users.AnyAsync(u => u.Email == email))                   // 2
        throw new BadRequestException("An account with this email already exists.");

    var user = new User
    {
        Name = dto.Name.Trim(),
        Email = email,
        PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password),          // 3
        Role = Roles.Member                                                    // 4
    };
    _db.Users.Add(user);
    await _db.SaveChangesAsync();                                             // 5

    return CreateAuthResponse(user);                                          // 6
}
```

1. **Normalize the email.** `John@Mail.com` and `john@mail.com` become the same account.
2. **Reject duplicates** with a clear message. This is a friendly check; the unique index in MySQL is the real guarantee if two requests arrive at the same moment (a race condition).
3. **Hash the password.** The plain password is never stored or logged (Part 7).
4. **Set the role on the server.** Every new account is a Member. Only an Admin can promote someone later.
5. **Save.** EF Core sends a parameterized `INSERT` to MySQL and fills in the new `Id`.
6. **Return a token** so the user is logged in right after registering.

PART 7

Password hashing with BCrypt

A **hash** is a one-way function: you can turn a password into a hash, but you cannot turn the hash back into the password. This is different from **encryption**, which can be reversed with a key.

	ENCRYPTION	HASHING
Direction	Two-way (decrypt with a key)	One-way
Used for	Data you must read back	**Passwords**
If the database leaks	Anyone with the key reads everything	Attackers only see hashes

What a BCrypt hash contains

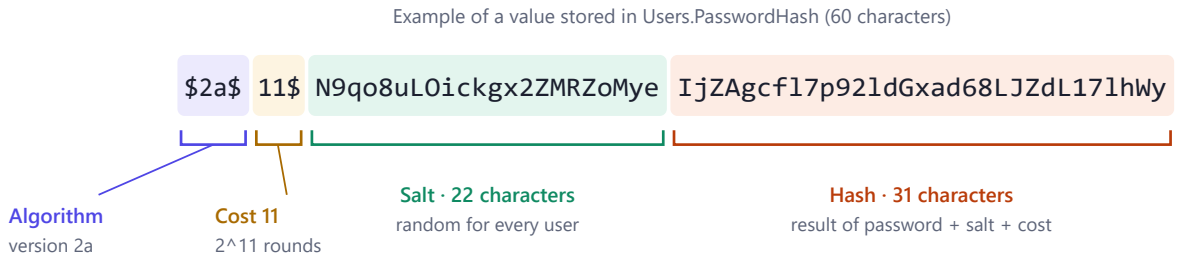


Figure 3. One BCrypt string stores everything needed to check a password later: version, cost, salt and hash. No separate salt column is needed.

Why the salt matters

Two users who both choose `Pass1234` get **different** hashes, because each gets a different random salt. Attackers cannot use a precomputed table of common passwords and their hashes (a **rainbow table**), and cannot see which users share a password.

Why BCrypt is deliberately slow

Fast hashes like SHA-256 let an attacker try billions of guesses per second. BCrypt repeats its work 2^{cost} times. BCrypt.Net uses cost **11** by default, which is what TaskFlow uses. Each +1 doubles the work for the server and for an attacker.

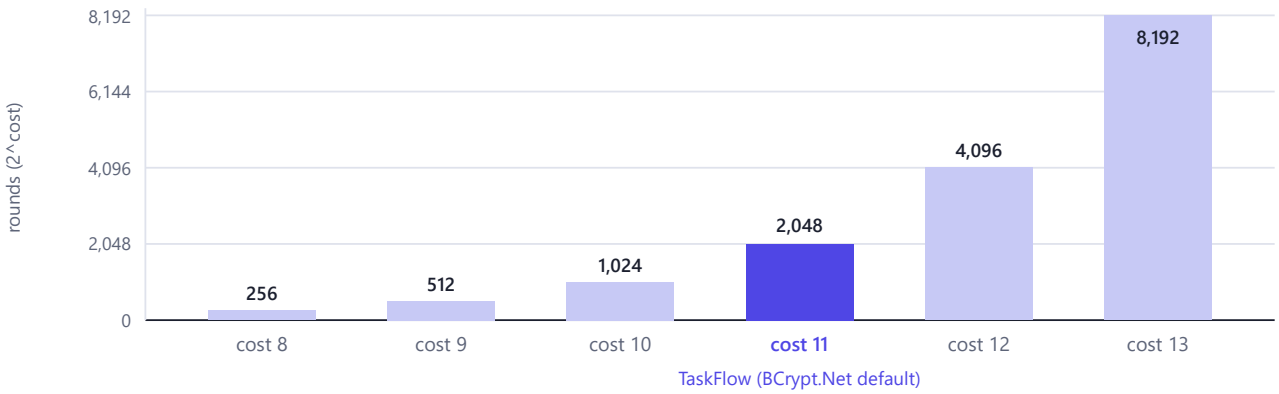


Figure 4. Rounds of work per password check for each cost factor. The number doubles with every step, so the cost can be raised as computers get faster.

How login checks a password

1. Read the stored hash from the database.
2. Take the version, cost and salt from the start of that hash.
3. Hash the password the user just typed with the **same** salt and cost.
4. Compare the two hashes. Same result → correct password.

All four steps happen inside one call: `BCrypt.Net.BCrypt.Verify(password, user.PasswordHash)`. The password is never decrypted, because it can't be.

PART 8

Login, step by step

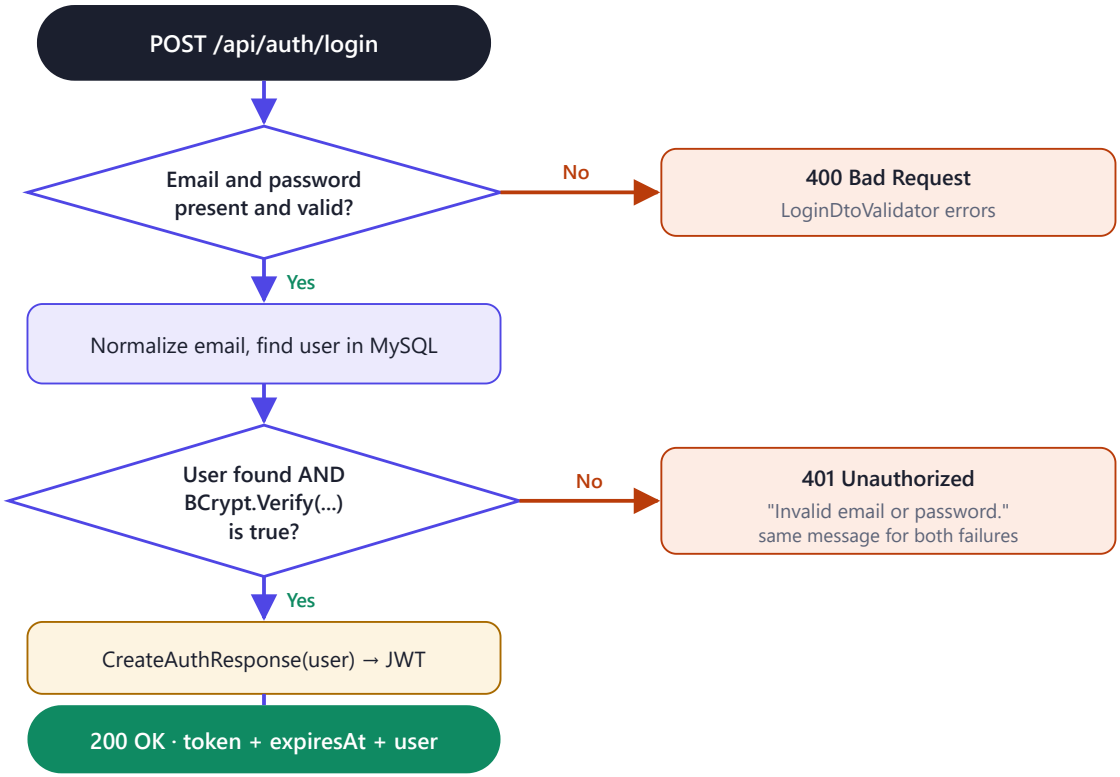


Figure 5. Login flow. A missing user and a wrong password give exactly the same 401 response.

```
public async Task<AuthResponseDto> LoginAsync(LoginDto dto)
{
    var email = dto.Email.Trim().ToLowerInvariant();
    var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == email);

    // Same message for both cases so attackers can't discover which emails exist
    if (user is null || !BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash))
        throw new UnauthorizedException("Invalid email or password.");

    return CreateAuthResponse(user);
}
```

User enumeration. If the API said “email not found” for one case and “wrong password” for the other, an attacker could test a list of emails and learn which ones have accounts. One generic message closes that gap.

PART 9

Creating the JWT

A **JSON Web Token** is a string with three parts separated by dots. TaskFlow builds it in the private method `CreateAuthResponse`, used by both register and login.



Figure 6. Structure of a TaskFlow token. Header and payload are only Base64Url encoded, so anyone can read them. Only the server can produce a valid signature.

```
private AuthResponseDto CreateAuthResponse(User user)
{
    var jwt = _config.GetSection("Jwt");
    var expires = DateTime.UtcNow.AddMinutes(jwt.GetValue<int>("ExpiresInMinutes")); // 240

    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Email, user.Email),
        new Claim(ClaimTypes.Name, user.Name),
        new Claim(ClaimTypes.Role, user.Role)
    };

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt["Key"]!));
    var token = new JwtSecurityToken(
        issuer: jwt["Issuer"],
        audience: jwt["Audience"],
        claims: claims,
        expires: expires,
        signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256));

    return new AuthResponseDto(new JwtSecurityTokenHandler().WriteToken(token), expires, user.ToDto());
}
```

The settings it reads (appsettings.json)

```
"Jwt": {
  "Key": "<long random secret, not shown here>",
  "Issuer": "TaskManager.API",
  "Audience": "TaskManager.Client",
  "ExpiresInMinutes": 240
}
```

PIECE	MEANING
Claims	Facts about the user. Later requests read the id and role from here without a database lookup.
Issuer (`iss`)	Who created the token: this API.
Audience (`aud`)	Who the token is meant for: the TaskFlow client.
Expiry (`exp`)	240 minutes after login. After that the token is rejected.
HMAC-SHA256	A **symmetric** algorithm: the same secret key signs and verifies.

> **Never put secrets in the payload.** The payload is encoded, not encrypted. Paste any token into jwt.io and you can read it. The signature does not hide data; it only proves nobody changed it.

PART 10

Checking the token on every request

After login, the client sends the token in a header on every request:

```
GET /api/projects
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

In React, the Axios request interceptor in `src/api/axios.js` adds this header automatically. In Swagger, the **Authorize** button does it.

The configuration (Program.cs)

```
var jwt = builder.Configuration.GetSection("Jwt");
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwt["Issuer"],
            ValidAudience = jwt["Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt["Key"]!)),
            ClockSkew = TimeSpan.Zero
        };
    });
builder.Services.AddAuthorization();
```

SETTING	WHAT IT CHECKS
`ValidateIssuerSigningKey`	The signature matches, so the token was not changed
`ValidateLifetime`	The token has not expired
`ValidateIssuer`	`iss` is `TaskManager.API`
`ValidateAudience`	`aud` is `TaskManager.Client`
`ClockSkew = Zero`	No grace period. .NET normally allows 5 extra minutes

Token creation (Part 9) and token checking (here) read the **same** `Jwt` section of `appsettings.json`. If the key, issuer or audience differed, every token would be rejected.

The middleware pipeline

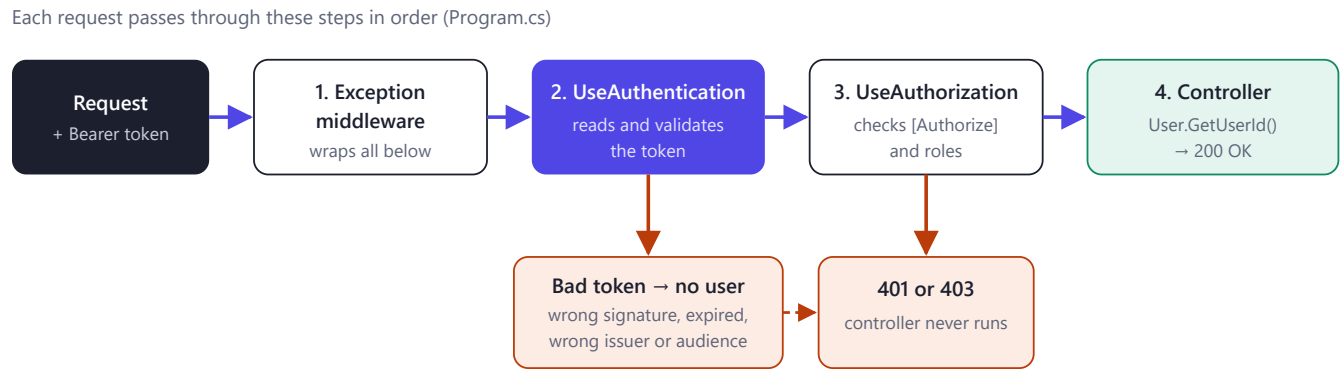


Figure 7. Authentication must run before authorization: the API has to know who the user is before it can check what they may do.

What happens inside UseAuthentication

1. Take the text after `Bearer` in the `Authorization` header. No header → the request continues as an anonymous user.
2. Recalculate the signature with the secret key and compare it.
3. Check expiry, issuer and audience.
4. Build a **ClaimsPrincipal** from the claims and put it in `HttpContext.User`. Controllers read it through the `User` property.

Reading the user in a controller

```
// Extensions/ClaimsPrincipalExtensions.cs
public static int GetUserId(this ClaimsPrincipal user) =>
    int.Parse(user.FindFirstValue(ClaimTypes.NameIdentifier)!);

public static bool IsAdmin(this ClaimsPrincipal user) => user.IsInRole(Roles.Admin);
```

> **The user id never comes from the request body.** The body is controlled by the client, so anyone could send someone else's id. The token is signed, so its id cannot be changed.

Token lifetime

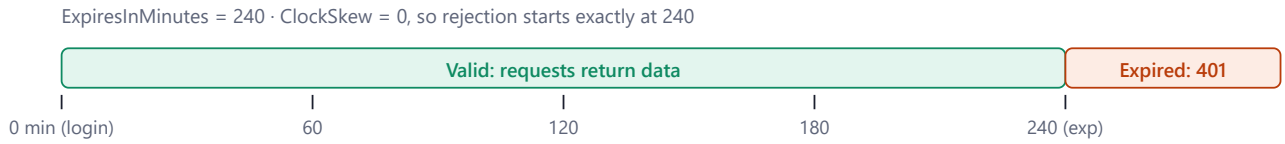


Figure 8. A token's life. After expiry the Axios response interceptor sees the 401, clears the session and sends the user to the login page.

PART 11

The /me endpoint

```
[Authorize]
[HttpGet("me")]
public async Task<ActionResult<UserDto>> Me() =>
    Ok(await _auth.GetCurrentUserAsync(User.GetUserId()));

// AuthService
public async Task<UserDto> GetCurrentUserAsync(int userId)
{
    var user = await _db.Users.FindAsync(userId)
        ?? throw new NotFoundException("User not found.");
    return user.ToDto();
}
```

The route has **no id in the URL**. The id comes from the token, so a user can only ever read their own record.

USE	WHY IT HELPS
Check a saved token on app start	200 means still valid; 401 means log in again
Load fresh user data	Reads the database, so it shows a role changed after login
Detect a deleted user	Returns 404 even if the token is still valid

Honest note: the React app defines `authApi.me()` but does not call it yet. It reads the user from localStorage. Calling `/me` when the app loads is a planned improvement.

PART 12

Errors and status codes

Services throw typed exceptions. `ExceptionMiddleware` catches the base type `AppException` and uses each subclass's `StatusCode` (polymorphism), so controllers need no `try/catch`.

```
public abstract class AppException : Exception { public abstract int StatusCode { get; } ... }
public class BadRequestException   : AppException { public override int StatusCode => 400; ... }
public class UnauthorizedException : AppException { public override int StatusCode => 401; ... }

// ExceptionMiddleware
catch (AppException ex) { await WriteError(context, ex.StatusCode, ex.Message); }
```

CODE	WHEN	RESPONSE BODY
200	Register or login succeeded	`{ token, expiresAt, user }`
400	Validation failed	`{ errors: { Password: ["..."] } }`
400	Email already registered	`{ status: 400, message: "An account with this email already exists." }`
401	Wrong email or password	`{ status: 401, message: "Invalid email or password." }`
401	Missing, changed or expired token on `/me`	Empty body (returned by the framework)
404	`/me` for a user who was deleted	`{ status: 404, message: "User not found." }`
500	Unexpected bug	Logged on the server; details hidden outside Development

On the React side, `getErrorMessage()` in `axios.js` reads either shape and shows it in a toast.

PART 13

Security checklist and limitations

What the code already does

PROTECTION	WHERE
Passwords hashed with salted BCrypt	`AuthService.RegisterAsync`
Generic login error (no user enumeration)	`AuthService.LoginAsync`
Role set by the server, never by the client	`Role = Roles.Member`, no Role in `RegisterDto`
Password strength rules	`RegisterDtoValidator`
Unique email (code check + database index)	`AuthService` + `AppDbContext`
Signed tokens with expiry, issuer and audience checks	`Program.cs`
No grace period after expiry	`ClockSkew = TimeSpan.Zero`
Password hash never returned	`UserDto` via `ToDto()`
SQL injection prevented	EF Core sends parameterized SQL

Known limitations and how to fix them

LIMITATION	RISK	IMPROVEMENT
Logout only clears the browser	A stolen token works until it expires	Short access tokens + refresh tokens stored in the database
Role is inside the token	A role change applies only after the next login (stale claims)	Shorter tokens, or re-check critical rights in the database
Token kept in localStorage	Readable by injected script (XSS)	httpOnly cookie with CSRF protection
No rate limiting on login	Password guessing	ASP.NET Core rate limiter, temporary lockout
Secret key in appsettings.json	Leaks if the file is shared	User Secrets, environment variables or a key vault
HTTP on localhost	Token readable on the network	HTTPS in production

> **Say it in the interview:** *The basics are in place: BCrypt hashing, signed expiring tokens and server-side roles. With more time I'd add refresh tokens, rate limiting on login and move the secret key out of the config file.*

PART 14

Testing it yourself

Run the API (`dotnet run` in `backend/TaskManager.API`) and open `http://localhost:5000/swagger`.

#	TRY THIS	EXPECTED
1	Register with password `abc`	400, three password errors
2	Register a new user with a valid password	200 with a token, role Member
3	Register again with the same email in capitals	400, email already exists
4	Login with a wrong password	401, “Invalid email or password.”
5	Login with an email that doesn’t exist	401, the same message
6	Call `GET /api/auth/me` without authorizing	401
7	Authorize with the token (without the word Bearer), call `/me`	200 with your user
8	Change one character of the token and call `/me`	401, signature no longer matches
9	Paste the token into jwt.io	You can read the payload: it isn’t encrypted
10	In MySQL: `SELECT Email, PasswordHash FROM Users;`	Hashes starting with `$2a$11$`

PART 15

Interview questions

1. Walk me through your login flow.

> The client posts email and password. I validate the input, normalize the email, load the user and check the password with BCrypt.Verify. On success I create a JWT with the user's id, name, email and role, signed with HMAC-SHA256, valid for 240 minutes. The client sends it as a Bearer token on every request.

2. How do you store passwords?

> As BCrypt hashes, never as plain text or encryption. Each hash has its own random salt and a cost factor of 11, so identical passwords give different hashes and brute force is slow.

3. Is a JWT encrypted?

> No. Header and payload are Base64Url encoded and anyone can read them. The signature only proves the token wasn't changed. That's why I never put sensitive data in it.

4. What stops a user from changing their role to Admin in the token?

> The signature. It's calculated from the header and payload with a secret key only the server knows. If the payload changes, the signature check fails and the API returns 401.

5. Why JWT and not sessions?

> JWT is stateless: the server stores nothing, so it scales across servers and suits a separate React frontend. The trade-off is that a token can't be revoked before it expires; refresh tokens solve that.

6. Why the same error for a wrong email and a wrong password?

> To prevent user enumeration, so attackers can't find out which emails are registered.

7. Why check for duplicate emails if there's a unique index?

> The code check gives a clear message. The index is the real guarantee when two requests arrive at the same moment, which the code check alone can't prevent.

8. Where does the controller get the user id?

> From the token's NameIdentifier claim through `User.GetUserId()`, never from the request body, because the body can be faked and the token can't.

9. Why does middleware order matter?

> UseAuthentication must run before UseAuthorization, because you need to know who the user is before checking their permissions. The exception middleware runs first so it can catch errors from everything after it.

10. What would you improve?

> Refresh tokens with server-side revocation, rate limiting on login, moving the secret key to environment variables, httpOnly cookies instead of localStorage, and calling /me on app start.

> **Say it in the interview:** *Authentication in my API works in two halves. Login checks the password with BCrypt and issues a signed JWT with the user's id and role. On every later request the JWT middleware validates the signature, expiry, issuer and audience and builds the ClaimsPrincipal, so my controllers know who is calling without touching the database.*